

FinResolve Edge

A real-time fintech incident-mapping and response platform secured with post-quantum cryptography.

The key is to give each technology a clear job, so the project does not feel like random buzzwords.

Technology	Its job in FinResolve Edge
Observability	Collect live payment, login, fraud, API, and system events
Edge computing	Detect and map incidents near the source with low delay
AI / graph analytics	Find connected users, devices, merchants, and transactions
Post-quantum cryptography	Secure incident events, response commands, and audit logs
Resolution engine	Recommend or execute safe response actions

How it works

Payment app / merchant / ATM / API

↓

Edge event collector

↓

Local anomaly detection + event filtering

↓

PQC-signed incident event is sent securely

↓

Central incident graph + observability dashboard

↓

Resolution playbook and analyst approval

What edge computing adds

Normally, every payment event goes to one central server. That can create delays during fraud spikes or outages.

With edge computing:

- **A merchant region, payment gateway, or branch gets a local edge node.**
- **The node detects suspicious behaviour immediately.**
- **It sends only important incidents to the central system.**
- **It can temporarily hold suspicious transactions even if the central cloud is slow or unavailable.**
- **It keeps mapping nearby fraud clusters in real time.**

Example:

Five accounts log in from one device in Delhi

↓

Delhi edge node detects the abnormal pattern

↓

It flags the device within seconds

↓

It sends a signed incident event to central FinResolve

↓

Central graph discovers the same recipient account across cities

↓

Analyst blocks the recipient and alerts affected users

Where post-quantum cryptography fits

PQC should protect the control plane, not be presented as the fraud-detection algorithm.

Use it for:

- **Signing incident alerts before they leave an edge node**
- **Verifying that an incident command really came from the central platform**
- **Protecting long-term audit logs**
- **Protecting sensitive fraud intelligence shared between branches or partners**

Use:

- **ML-DSA-65 to sign incident events and resolution commands**
- **ML-KEM-768 to establish a secure key between edge node and central platform**
- **AES-256-GCM to encrypt event payloads**
- **SHA-384 for tamper-evident audit-log hashes**

Observability layer

Collect four event types:

1. **Transaction events**
Amount, merchant, timestamp, payment status, recipient, location.
2. **Authentication events**
Login attempts, device ID, IP address, failed OTP attempts.
3. **System events**
API latency, service errors, database health, failed payment gateway calls.
4. **Security events**
Fraud alerts, unusual device behaviour, blocked transfers, suspicious recipients.

Use OpenTelemetry-style event collection and real-time WebSockets for dashboard updates.

Incident-mapping engine

The system creates a graph:

Account → Device → IP Address → Transaction → Merchant → Recipient

It then detects shared links, such as:

- **One device controlling many accounts**
- **One recipient receiving money from many flagged accounts**
- **A merchant causing a sudden increase in failed payments**
- **A payment API outage affecting a particular city or region**

Resolution playbooks

Incident	Automated recommendation
Suspicious device cluster	Block session and require re-authentication
Recipient fraud cluster	Hold pending payments and flag recipient
Payment gateway outage	Route payments to backup gateway
Merchant failure spike	Alert merchant and reduce transaction retries
Regional API issue	Route traffic to another healthy service region

Keep irreversible actions human-approved in the demo. For example, QShield can recommend freezing a payment, but an analyst must click Approve.

MVP scope

Build only these components:

- **Fake fintech transaction generator**
- **One simulated edge node**
- **Real-time event dashboard**
- **Incident map and graph**
- **One anomaly detector**
- **Three resolution playbooks**
- **PQC signature verification for edge-to-cloud incident messages**

Live demo

- 1. Generate normal payments.**
- 2. Trigger a fake fraud attack: many accounts, one device, one recipient.**
- 3. Edge node detects the cluster immediately.**
- 4. Incident appears on the dashboard and relationship graph.**
- 5. The central platform verifies the PQC-signed event.**
- 6. FinResolve recommends holding the recipient's pending transfers.**
- 7. Analyst approves the action.**
- 8. Dashboard shows the incident as contained.**

Pitch line:

“FinResolve Edge detects financial incidents where they begin, maps their spread in real time, and helps fintech teams contain them before they become losses.”

FinResolve Edge: complete tech flow

Simulated Fintech Events



Edge Collector



Local Detection Engine



PQC-Signed Incident Event



Central Event Stream + Observability Store



Incident Mapper and Risk Engine



Response Playbook Engine



Analyst Dashboard and Approval

1. Simulated fintech events

Generate realistic fake events:

- Payment created / completed / failed
- Login succeeded / failed
- OTP failure
- New device login
- Merchant API latency or outage
- Multiple transfers to one recipient
- Same device accessing many accounts

Each event includes:

```
{  
  "event_id": "evt_001",  
  "event_type": "payment",  
  "account_id": "acc_123",  
  "device_id": "dev_45",  
  "ip_address": "203.0.113.20",  
  "merchant_id": "mer_12",  
  "recipient_id": "rec_90",  
  "amount": 25000,  
  "city": "Delhi",  
  "timestamp": "2026-09-13T10:20:00Z",  
  "status": "completed"  
}
```

2. Edge collector

Simulate one local edge node, for example Delhi Edge Node.

Its role:

- Receives local events first
- Filters normal events
- Detects urgent patterns quickly
- Sends only high-risk incident events to the central platform
- Continues working if the central service is temporarily unavailable

3. Local detection engine

Start with simple and explainable rules.

Rule 1: More than 3 failed logins from one device in 5 minutes

→ Suspicious login incident

Rule 2: One device used by more than 3 accounts in 10 minutes

→ Account-takeover cluster

Rule 3: More than 5 payments to one recipient from flagged accounts

→ Mule-account / recipient fraud incident

Rule 4: Payment API error rate above 20%

→ Payment-service outage incident

Rule 5: Merchant transaction failures rise sharply

→ Merchant-side operational incident

Optional AI layer:

- **Isolation Forest** for unusual transaction amounts or frequency
- **DBSCAN** for location/time-based clusters
- **NetworkX graph analytics** to find shared devices, IPs, merchants, and recipients

4. PQC security layer

Before the edge node sends an incident to the cloud:

- Sign the incident with **ML-DSA-65**
- Establish a protected session through **ML-KEM-768**
- Encrypt the payload using **AES-256-GCM**
- Hash audit logs using **SHA-384**

This proves:

- The incident event came from a trusted edge node
- Nobody changed it during transit
- The response command came from the real central system

5. Central incident mapper

The central platform stores the relationships:

Account → Device → IP → Transaction → Merchant → Recipient

Use a graph database to expose connected fraud patterns.

Example:

Device D-45

└─ Account A-101

└─ Account A-202

└─ Account A-303

└─ Recipient R-900

The system groups these into one incident instead of creating many unrelated alerts.

6. Response playbook engine

Incident	Suggested action
Suspicious device	End session and require re-login
Account takeover cluster	Temporarily hold transactions
Mule recipient	Hold pending transfers and flag recipient
Payment gateway outage	Route requests to backup gateway
Merchant failure spike	Alert merchant and reduce retry traffic

For the hackathon, the system should only **recommend** high-impact actions. The analyst approves them from the dashboard.

7. Dashboard

Build five simple sections:

- Live event feed
- Incident severity cards
- Geographic map of incident locations
- Relationship graph of affected entities
- Response panel with approve/reject action

Recommended stack

Layer	Technology
Frontend	React, Vite, Tailwind CSS

Layer	Technology
Dashboard charts	Recharts
Relationship graph	React Flow or Cytoscape.js
Map	MapLibre GL JS or Leaflet
Backend	FastAPI
Database	PostgreSQL
Graph analysis	NetworkX initially
Optional graph database	Neo4j
Live updates	WebSockets
Edge simulation	Separate FastAPI service
PQC	liboqs with ML-KEM and ML-DSA
Classical encryption	Python cryptography
Observability	OpenTelemetry, structured JSON logs
Deployment	Docker Compose